ESW Report: Image Inpainting On the Edge

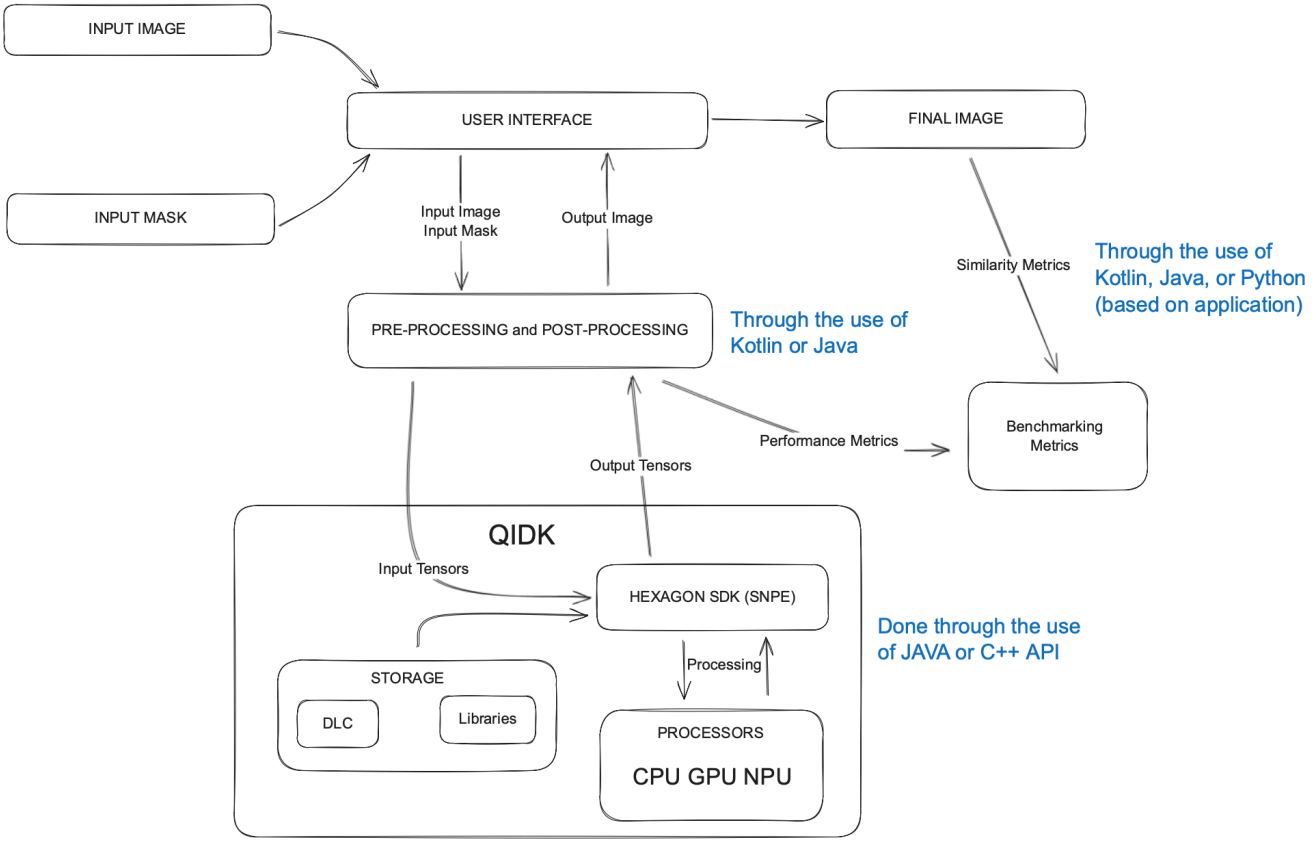
Team 26: World Domination
Vikesh Kansal [2024101126]
Kaushik Yadala [2024101123]
Srinivas Kolla [2024101146]
Anivarth Penumetcha [2024111006]

Problem Stament

- To learn and apply the concepts of edge computing.
- Use the Qualcomm innovators development kit as a springboard to dive into the world of edge computing.
- Using state of the art hardware provided by Qualcomm to run AI models, especially image inpainting models, at the edge, using the CPU, GPU and the NPU (Hexagon Tensor Processor) or DSP (Digital Signal Processor).
- Get a taste of the exhilarating and enthralling world of embedded systems.

Methodology

Block Diagram



Framework

- Qualcomm SDK
  - Snapdragon Neural Processing Engine (SNPE)
    - TF-Lite Delegate
    - Snpe-net-run
    - Snpe-dlc-quant
  - Android Framework
    - Java for routing and logic
    - Java for the TF-Lite Delegate API
    - Gradle for management of dependencies
    - XML for UI
  - Python for similarity metrics and router model creation

Parameters Measured

Performance Metrics

- Inference Time
- Memory Usage
- NPU/GPU/CPU Usage
  Accuracy Metrics
- Peak Signal to Noise Error
- Structural Similarity Index Measure
- Learned Perpetual Image Patch Similarity
- Frechet Inception Distance

Models Use

Qualcomm AI Hub Models

- AOT-GAN
- LaMa Dialated

Complete NPU Workflow

---

1. Application Workflow
1.1 Application Flow:

1. **User Interface Setup** (MainActivity.onCreate)
   - The app initializes with two spinners: one for **model selection** (AOT-GAN, LaMa, Hi-Fill) and one for **runtime selection** (NPU, GPU, CPU)
   - Users can select a source image and a mask image from their device gallery
   - The "Run Inpainting" button becomes enabled only when both images are selected and a model is loaded
2. **Model Loading Process** (loadModelAsync)
   - When user selects both a model and runtime, the app triggers asynchronous model loading
   - The process runs on a **background thread** (ExecutorService) to avoid blocking the UI
   - Based on the selected model, the appropriate `.tflite` file is retrieved from app assets:
     - `aot_gan.tflite` for AOT-GAN

- `lama_dilated.tflite` for LaMa
- `model_float32.tflite` for Hi-Fill

3. **Delegate Priority Configuration**
   - The app creates a **delegate priority order** based on runtime selection:
     - **NPU**: Attempts QNN NPU delegate first, falls back to CPU if initialization fails
     - **GPU**: Attempts GPUv2 delegate first, falls back to CPU if initialization fails
     - **CPU**: Uses XNNPack (optimized CPU execution)
4. **Inference Execution** (updatePredictionDataAsync)
   - User clicks "Run Inpainting" button
   - Inference runs on background thread
   - Results are posted back to UI thread for display

---

2. TensorFlow Lite Integration and Inference Pipeline

2.1 Model Loading (TFLiteHelpers.java)

The `TFLiteHelpers` class implements a sophisticated model loading system:

```
Key Steps:
1. Load .tflite model file from assets into MappedByteBuffer
2. Calculate MD5 hash of model for caching purposes
3. Create hardware acceleration delegates (QNN NPU, GPUv2, or CPU)
4. Configure interpreter with delegate priority order
5. Allocate tensors for input/output
```

Delegate Creation Strategy:

- The app attempts to create delegates in a **priority order**
- If a delegate fails to initialize, it tries the next option
- This ensures the app always runs, even if specific hardware is unavailable

2.2 Model Validation (ImageInpainter constructor)

The ImageInpainter validates the loaded model has the correct structure:

- **2 input tensors**:

- Input 0: RGB image [1, Height, Width, 3]
- Input 1: Grayscale mask [1, Height, Width, 1]

- **1 output tensor**:
  - Output 0: Inpainted RGB image [1, Height, Width, 3]

---

3. How Images are Processed using TF-Lite

3.1 Preprocessing (preprocess method)

Step 1: Image Resizing

```
- Source image and mask are resized to match model's expected
input dimensions
- Aspect ratio is maintained using padding
(ImageProcessing.resizeAndPadMaintainAspectRatio)
- Padding ensures no distortion in images
```

Step 2: Pixel Extraction and Normalization

```
// For RGB Image (3 channels):
for each pixel in resizedImage:
    R = ((pixelValue >> 16) & 0xFF) / 255.0f  // Normalize to
[0,1]
    G = ((pixelValue >> 8) & 0xFF) / 255.0f
    B = (pixelValue & 0xFF) / 255.0f

// For Mask (1 channel - grayscale):
for each pixel in resizedMask:
    maskValue = ((pixelValue >> 16) & 0xFF) / 255.0f  // Use red
channel
```

Step 3: Buffer Creation

```
- Creates ByteBuffer with native byte order for optimal
performance
```

```
- Allocates: 1 × Height × Width × Channels × 4 bytes (float32)
- Buffers are populated in row-major order (HWC format)
```

3.2 TF-Lite Inference Execution (inpaintImage method)

```
// 1. Prepare inputs
Object[] inputs = {imageInputBuffer, maskInputBuffer};

// 2. Prepare output buffer
ByteBuffer outputBuffer = ByteBuffer.allocate(1 × H × W × 3 ×
4);
Map<Integer, Object> outputs = new HashMap<>();
outputs.put(0, outputBuffer);

// 3. Run inference
tfLiteInterpreter.runForMultipleInputsOutputs(inputs, outputs);
```

What happens during inference:

- The TF-Lite interpreter distributes operations across available hardware
- Operations compatible with NPU/GPU run on those accelerators
- Unsupported operations automatically fall back to CPU (XNNPack)
- The interpreter manages memory and execution automatically

3.3 Postprocessing (postprocess method)

```
// 1. Convert ByteBuffer to FloatBuffer for easier access
FloatBuffer floatOutputBuffer = outputBuffer.asFloatBuffer();

// 2. Denormalize and convert to pixel values
for each pixel in output:
    R = floatOutputBuffer.get() × 255.0f  // Denormalize from
[0,1] to [0,255]
    G = floatOutputBuffer.get() × 255.0f
    B = floatOutputBuffer.get() × 255.0f

    // Clamp values to valid range
    R = clamp(R, 0, 255)
```

```
    G = clamp(G, 0, 255)
    B = clamp(B, 0, 255)

    pixel = Color.rgb(R, G, B)

// 3. Create Android Bitmap from pixel array
resultBitmap.setPixels(pixels, ...)
```

---

4. How Everything Runs on the NPU

4.1 Qualcomm QNN (Qualcomm Neural Network) SDK Integration

The app uses the **QNN Delegate** to enable NPU acceleration on Qualcomm chipsets:

Dependencies (build.gradle):

```
implementation "com.qualcomm.qti:qnn-runtime:2.37.0"
implementation "com.qualcomm.qti:qnn-litert-delegate:2.37.0"
```

Native Libraries (AndroidManifest.xml):

```
<uses-native-library android:name="libcdsprpc.so"
android:required="false"/>
```

- `libcdsprpc.so` : Qualcomm's DSP (Digital Signal Processor) RPC library for NPU communication

4.2 QNN Delegate Configuration (CreateQNN_NPUDelegate)

Hardware Capability Detection:

```
if (QnnDelegate.checkCapability(Capability.DSP_RUNTIME)) {
    // Use DSP backend for older Qualcomm chips
    qnnOptions.setBackendType(BackendType.DSP_BACKEND);
    qnnOptions.setDspOptions(DSP_PERFORMANCE_BURST,
DSP_PD_SESSION_ADAPTIVE);
} else {
    // Use HTP (Hexagon Tensor Processor) for newer chips
```

```
    qnnOptions.setBackendType(BackendType.HTP_BACKEND);
    qnnOptions.setHtpPerformanceMode(HTP_PERFORMANCE_BURST);
    qnnOptions.setHtpPrecision(HTP_PRECISION_FP16);  // FP16 for
faster execution
}
```

Model Compilation and Caching:

```
qnnOptions.setSkelLibraryDir(nativeLibraryDir);  // Path to QNN
native libraries
qnnOptions.setCacheDir(cacheDir);                 // Cache for
compiled models
qnnOptions.setModelToken(modelIdentifier);        // Unique hash
for this model
```

- **First run**: QNN compiles the TF-Lite model to NPU-optimized format (takes time)
- **Subsequent runs**: QNN loads pre-compiled model from cache (much faster)

Performance Optimization:

```
qnnOptions.setHtpPerformanceMode(HTP_PERFORMANCE_BURST);
```

- **BURST mode**: Maximum performance at cost of battery/heat
- Suitable for one-time inference tasks

4.3 Delegate Priority and Heterogeneous Execution

Key Innovation: Heterogeneous Computing

```
// Delegate registration order determines operation assignment
priority
tfLiteOptions.addDelegate(qnnNPUDelegate);  // Priority 1: NPU
gets first pick
tfLiteOptions.addDelegate(gpuDelegate);      // Priority 2: GPU
gets remaining ops
tfLiteOptions.setUseXNNPACK(true);          // Priority 3: CPU
fallback always enabled
```

How TF-Lite distributes work:

1. **QNN NPU Delegate** analyzes the model graph
2. Operations supported by NPU (convolutions, pooling, etc.) are assigned to NPU
3. Unsupported operations (certain activations, dynamic shapes) are left for GPU/CPU
4. **GPUv2 Delegate** picks up remaining GPU-compatible operations
5. **XNNPack** runs any operations not handled by NPU/GPU

This creates a **heterogeneous execution** where different parts of the model run on different hardware accelerators simultaneously.

---

5. GPU Acceleration (Alternative Runtime)

If GPU runtime is selected:

GPUv2 Delegate Configuration:

```
gpuOptions.setInferencePreference(INFERENCE_PREFERENCE_SUSTAINED_SPEED);
gpuOptions.setPrecisionLossAllowed(true);  // Enable FP16 for faster execution
gpuOptions.setSerializationParams(cacheDir, modelIdentifier); // Cache optimization
```

Native Library:

```
<uses-native-library android:name="libOpenCL.so" android:required="false"/>
```

- Uses OpenCL for GPU compute operations

---

6. Thread Management and Asynchronous Execution

Background Thread Executor:

```
ExecutorService backgroundTaskExecutor = Executors.newSingleThreadExecutor();
```

- All heavy operations (model loading, inference) run on background thread
- Prevents UI freezing and ANR (Application Not Responding) errors

Main Thread Handler:

```
Handler mainLooperHandler = new Handler(Looper.getMainLooper());
```

- Results posted back to main UI thread for display updates
- Required by Android - UI updates must happen on main thread

Execution Flow:

```
User Action → Background Thread (Inference) → Main Thread (UI
Update)
```

---

7. Performance Monitoring

```
long inferenceTime =
tfLiteInterpreter.getLastNativeInferenceDurationNanoseconds();
```

- TF-Lite provides native inference time measurement
- Excludes preprocessing/postprocessing time
- Displayed to user in milliseconds

Failed/Unused Workflows

Workflow 1 (on app)

- Pre-process image and mask to the respective input tensors
- Run the DLC on the provided input tensors
- Post-process the output tensor into an output image
- Output the in-painted image, and performance metrics to the screen.
- Take the ideal image, and the inpainted image and pass it to python (on laptop) to get similarity scores
  Workflow 2 (using SNPE)

- Pre-process image and mask to the respective raw files
- Run the DLC on the provided input tensors, using snpe-net-run and snpe-bench.py.
- Post-process the output raw into an output image
- Get the csv file from snpe-bench.py
- Take the ideal image, and the inpainted image and pass it to python to get similarity scores

Dataset

As there were no ready to use image datasets for our purposes, we created a dataset of 101 sets of 3 images (original image, mask, and ideal image). We obtained images from the Stanford Image Inpainting datasets, specifically, CelebaHQ and Places 365. We then created masks of each of the images using GIMP of the objects that we wanted to inpaint. We then used state of the art inpainting models such as Qwen Image Edit to inpaint the image to give us an ideal image to compare our model to. This process allowed us to create a dataset with 101 sets of images. Each set of images contained an original, pre-inpainted image from the Stanford dataset, a corresponding mask, and an ideal image from Qwen.
Example:

Classification of Images

Each of the image from the dataset was classified into different categories as follows:

- Resolution & aspect ratio - Basic image dimensions
- Face detection - Uses MediaPipe to count faces
- Texture analysis - Laplacian variance (blur detection) and edge density
- Color analysis - Brightness, saturation, and dominant colors via k-means clustering
- Text detection - Uses Tesseract OCR if available

- CLIP embeddings - Semantic content understanding using OpenAI's CLIP model
- Mask analysis: analyzes mask area, bounding box, location, and fragmentation
  This was done through the creation of a python script which utilized the libraries and models available within python, as mentioned above.
  Our classification results are detailed below:

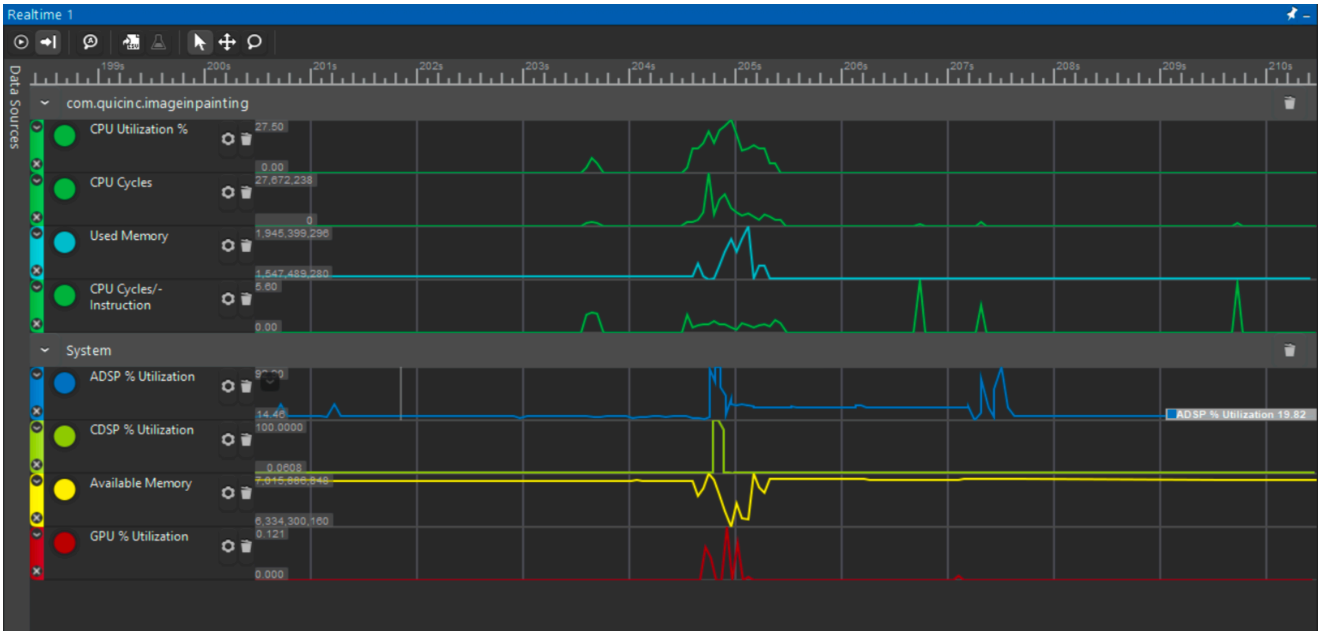
Category	Sub-Category	Image Count
Complexity	High Complexity	57
	Medium Complexity	21
	Low Complexity	24
Brightness	Very Bright	2
	Bright	37
	Medium	56
	Dark	7
Saturation	High Saturation	26
	Medium Saturation	52
	Low Saturation	24
Mask Coverage	Medium Coverage	25
	Small Coverage	77
Mask Location	Center	51
	Edge/Corner	51
Mask Fragmentation	Single Region	97
	Few Regions	5
Content Type	Contains Faces	13
	Contains Text	7
	General	83

Benchmarking

Runtime Analysis

To test the runtimes of our models, we measured the time it took for the model to run on different processors, and also utilized the snapdragon profiler to get more specific results. The resulting runtimes for the models is as follows:

		Average Inference Time Results
LaMa Dilated	NPU	70ms
	GPU	250ms
	CPU	6500ms
AOT-GAN	NPU	130ms
	CPU	300ms
	GPU	8200ms



Similarity Analysis

To determine the effectiveness of AOT-GAN and LaMa Dilated while inpainting we created a python script that ran the following similarity metrics on the resulting image of the models and the ideal image we created in our dataset:

- *Learned Perceptual Image Patch Similarity* (LPIPS):
  - Chosen as it shows a subjective similarity of what human think is similar as opposed to what a computer would think is similar.
  - Utilizes a tuned model based on Inception V3
- *Frechet Inception Distance* (FID):

    - This measures the diversity of the images created by the model, allows us to see how varied the results of the images are.
- *Structural Similarity Index Measurement* (SSIM):
    - This measures a perceptual metric that quantifies image quality degradation.
    - It tries to quantify the similarity of an images as a human eye would percieve it, however it is not as effective as LPIPS.
- *Peak Signa-to-Noise ratio* (PSNR):
    - It measures how different the pixels are and hence is not good at defining of images are similar to a human, but gives us a quantified idea on how the images could be similar, at least to a computer.
      We ran the similarity test across whole images and also within the mask to see how they compare to each other.

These were the results of our analysis:

Model	Similarity Metric	Complete Image	Only Mask
LaMa Dilated	LPIPS	0.1826	0.0733
	SSIM	0.7621	0.7636
	PSNR	21.1723	14.7645
	FID	135.9633	135.9633
AOT-GAN	LPIPS	0.1903	0.0750
	SSIM	0.7579	0.7591
	PSNR	21.1251	15.1314
	FID	138.4853	138.4853

As can be seen from the above results, the results from Mask seem to show more similarity in LPIPS, and we think this is because the human eye is better at identifying small changes when the whole image is in context as opposed to only the mask area which may look similar.

Model Selector

To optimize the inpainting pipeline, we developed a lightweight Convolutional Neural Network (CNN) to act as a "Router." Instead of running all inpainting models on every

image, this Router analyzes the input image and the mask to predict which model (AOT-GAN or LaMa) is likely to yield a higher quality result.

Model Architecture Selection

We selected **ResNet18** as our backbone architecture as it offers an ideal balance of depth and parameter efficiency (~11 million parameters), allowing it to generalize well on smaller datasets while remaining computationally inexpensive for edge deployment.

Architectural Adaptations

Standard ResNet models are pre-trained on ImageNet and expect a 3-channel input (Red, Green, Blue). However, for inpainting tasks, the **mask** is just as important as the image itself, as it defines the context of the damage. To accommodate this, we modified the network structure:

1. **4-Channel Input Layer:** We conceptually "stacked" the binary mask onto the RGB image to create a 4-channel tensor (H×W×4). We surgically removed the first convolutional layer of the pre-trained ResNet18 and replaced it with a new layer accepting 4 input channels.
2. **Smart Weight Initialization:** Instead of initializing the new input layer with random noise (which would destroy the pre-trained feature detectors), we utilized a mean-weight transfer strategy. We copied the weights from the original RGB channels and averaged them to initialize the weights for the new Mask channel. This allowed the model to retain its ability to detect edges and textures from the very first training epoch.
3. **Binary Classification Head:** We truncated the final fully connected layer (originally designed to classify 1000 ImageNet categories) and replaced it with a single output neuron. This neuron outputs a logit that is passed through a Sigmoid activation function to provide a probability score:
   - **P<0.5:** AOT-GAN is preferred.
   - **P>0.5:** LaMa Dilated is preferred.

Training Methodology

The model was trained using **Transfer Learning**. We utilized the "Ideal" images created in our dataset generation phase to simulate the input context.

- **Label Generation:** Ground truth labels were generated automatically by comparing the LPIPS scores of AOT-GAN and LaMa. If LaMa had a lower (better) LPIPS score for a specific image, that image was labeled `1`, otherwise `0`.

- **Data Augmentation:** To prevent overfitting on the 101-image dataset, we applied synchronized random horizontal flips and rotations to both the image and the mask during training.
- **Loss Function:** We utilized Binary Cross Entropy with Logits (`BCEWithLogitsLoss`) to optimize the model.
  The Evaluation of the Model we created:

	Precision	Recalls	F1-Score	Support
AOT-GAN	0.88	0.64	0.74	11
LaMa Dialated	0.69	0.90	0.78	10
Accuracy			0.76	21
Macro Avg	0.78	0.77	0.76	21
Weighted Avg	0.79	0.76	0.76	21

Conclusion:

This successfully demonstrated the feasibility and efficiency of deploying state-of-the-art generative AI on edge devices. By leveraging the **Qualcomm QIDK and the Hexagon Tensor Processor (NPU)**, we achieved a dramatic reduction in inference time—dropping from 6500ms on the CPU to just 70ms on the NPU for LaMa Dilated.
Beyond raw performance, our project introduced a novel **"Router" architecture** based on a modified ResNet18. This intelligent selection mechanism allows the system to dynamically choose between AOT-GAN and LaMa based on image complexity, optimizing for the highest visual fidelity (LPIPS) without user intervention. Through the creation of a custom dataset and the implementation of a heterogeneous computing pipeline, Team "World Domination" has shown that complex image inpainting tasks can be executed entirely on-device, ensuring user privacy, low latency, and independence from cloud resources.